TAKE-HOME TECHNICAL ASSIGNMENT

Patient Intake & Triage Copilot

Senior AI Engineer — Agentic Systems & Applied ML

Estimated effort	8–12 focused hours, within a 5–7 day window
Format	Take-home build, followed by a 60–90 minute technical deep-dive
Tech stack	Your choice — Python at the core
Submission	Public GitHub repository (details at the end)

How to read this brief

This is written as a product brief, not a specification.

We describe the problem and the outcomes we care about, but we have deliberately left the *how* to you. We want to see how a senior engineer scopes ambiguity, makes trade-offs, and ships something real end-to-end. There is no single correct architecture, model choice, or framework. We care far more about your judgment, the quality of your engineering, and the clarity of your reasoning than about how many features you cram in.

A NOTE ON TIME — PLEASE READ

We do **not** expect a production system in 8–12 hours. We expect a thoughtfully scoped slice that genuinely works, plus a clear account of what you would do with more time. **Optimize for depth over breadth.** If you find yourself spending much more than ~12 hours, you have over-built — stop, and write up the rest as "what I'd do next." A smaller, sharply-reasoned submission beats a sprawling, shallow one.

Background: why this problem matters

NO US-HEALTHCARE EXPERIENCE REQUIRED

You do **not** need any prior background in US healthcare. Everything you need is in this brief, including a plain-language glossary and a worked example below. We are testing your engineering and AI judgment and how quickly you can get up to speed in an unfamiliar domain — **not** your domain knowledge. The clinical concepts here are intentionally simple ("chest pain → emergency").

A large share of visits to US emergency departments are for problems that could be handled safely — and far more cheaply — in a lower-acuity setting such as primary care, urgent care, or a telehealth visit. At the same time, some symptoms genuinely are emergencies and need care within minutes. Getting patients to the *right level of care at the right time* is called **triage**, and health systems, insurers, and virtual-first providers invest heavily in nurse triage lines and "care navigation" to do it well.

It is a hard balance. **Under-triage** (sending someone home who needed the ER) risks patient harm; **over-triage** (sending everyone to the ER) wastes resources and overwhelms hospitals. A well-designed AI copilot can help front-line staff and patients gather the right information and reach a safe, well-reasoned recommendation faster — which is exactly the kind of system we build.

Key terms (no prior healthcare knowledge assumed)

Term	What it means for this assignment
Triage	Sorting people by urgency so each gets the right level of care at the right time.
Care level / disposition	The recommended next step, e.g.: self-care at home · see primary care · telehealth visit · urgent care · emergency (ER / call 911).
Red-flag symptom	A feature signalling a possible emergency that needs immediate escalation (e.g., chest pain with shortness of breath).
Triage protocol / guidance	The rule set that maps a patient's symptoms and answers to a recommended care level.
Primary care / urgent care / ER	Routine ongoing care · same-day care for non-life-threatening issues · the hospital emergency department for serious or life-threatening problems.
Safety-netting	Telling the patient which worsening signs should prompt them to seek higher-level care.
PHI	Protected health information — real patient data. Must **never** be used here; synthetic only.

The problem we want you to solve

Build an agentic **"Patient Intake & Triage Copilot."** A patient (or a front-desk staff member on their behalf) describes their symptoms in plain language. Your copilot gathers the information it needs, reasons about urgency using triage guidance, and recommends an appropriate **care level** — clearly explaining why, and escalating immediately if it sees a possible emergency.

CRITICAL FRAMING — DESIGN FOR THIS

This is a **care-navigation and decision-support tool, not a diagnosis or treatment tool**. It must not name a diagnosis or recommend specific prescription medications, and a human (clinician or the patient's own judgment) owns the final decision. When uncertain, it should **bias toward caution** — recommend a higher level of care, never a lower one — and it must never delay an emergency by asking unnecessary questions. Build it grounded, transparent, and safe.

Inputs you can assume (you define the exact shapes)

- A patient's **presenting complaint** in free text (e.g., "I've had a sore throat and fever for three days").
- An optional **basic profile** — e.g., age, sex, key conditions, current medications (plain text), pregnancy status. You decide what you collect and how.

- **Triage guidance** — synthetic protocols you author, modeled on the *structure* of standard triage logic (symptom → questions to ask → red flags → recommended care level). Do not copy any proprietary protocol verbatim.

What the copilot should do (capabilities, not prescribed steps)

1. **Gather the right information.** Ask focused follow-up questions to fill the gaps needed to triage safely (duration, severity, associated symptoms, relevant history). Know when it has enough to decide — and when to stop asking.
2. **Catch emergencies first.** Detect red-flag presentations and escalate immediately and unambiguously, ahead of any routine questioning.
3. **Recommend a care level.** Map the situation to a disposition using the guidance, reasoning criterion by criterion.
4. **Explain and cite.** Every recommendation must cite the specific reported facts and the specific guidance rule it relied on. Include plain-language safety-netting ("seek care sooner if…").
5. **Stay in its lane.** No diagnosis, no specific medication or dosing advice; a calm, clear, non-alarming tone; appropriate disclaimers.
6. **Know its limits.** Under uncertainty or for out-of-scope input, say so and route to a human; default to the safer (higher) level of care.

A worked mini-example (illustrative — not a spec)

To make the task concrete, here are two tiny synthetic scenarios. They show the *kind* of behavior we mean; they do not prescribe how to build it.

Scenario	Expected copilot behavior
Emergency. 58-year-old with a history of high blood pressure: "Crushing chest pain for 30 minutes, short of breath and sweaty."	Immediately recognizes a red-flag pattern → disposition = **EMERGENCY: call 911 now**. Cites the reported facts plus the red-flag rule. Does *not* keep asking routine questions, and does *not* attempt a diagnosis.
Low acuity. 25-year-old: "Mild sore throat since yesterday, no fever, no trouble breathing or swallowing."	Asks a couple of targeted questions, then recommends **self-care** (or primary care if it persists), with safety-netting: seek care if breathing/swallowing difficulty or high fever develops. Cites the facts and the rule used.

If a needed fact is missing — say, the duration or whether breathing is affected — the copilot should **ask** for it rather than assume, and if it still cannot tell, escalate to the safer option.

THIS IS INTENTIONALLY AN AGENTIC PROBLEM

We chose a problem that is hard to solve well with a single LLM call. Expect to design a system that decides what to ask, gathers information over multiple turns, consults guidance, applies a safety check, and decides when it is done or must escalate. **How** you orchestrate that — an agent framework, a state machine, a custom controller, or something else — is your decision. Be ready to defend it.

Engineering expectations (the bar)

- **Python at the core.** Clean, idiomatic, well-structured code; typed where types earn their keep.
- **A real interface.** At minimum a (likely conversational) API with sensible endpoints. A thin chat UI or CLI is welcome but optional.
- **Tests that matter.** Especially around the safety-critical paths — the tests a senior writes to trust that emergencies are caught.
- **Operability.** Configuration, logging, and error handling fit to hand to a teammate. No secrets in the repo. Reproducible setup (containerized or a one-command run).
- **Scale & cost awareness.** Be ready to discuss: how does this behave at 50,000 conversations/day? What is your per-conversation latency and cost story for a chat workload?

AI / ML expectations (the senior bar)

- **Model choice is a decision, not a default.** Use whatever models you like — hosted frontier, small / open-weight, or local — and reason about which model for which sub-task and the cost / latency / quality / privacy trade-offs, including where a small or local model is the right call for sensitive data.
- **Don't trust the LLM alone for safety.** Emergency detection is the highest-stakes path. We are very interested in how you make it reliable (e.g., a deterministic safety net alongside the model) and how you measure it.
- **Evaluation is not optional.** Include an eval harness and a labeled set of cases — with adversarial / edge cases such as patients downplaying symptoms, ambiguous or mixed signals, and out-of-scope input. Show how you would catch a regression, and pay special attention to *under-triage* (missed emergencies).
- **Observability, grounding & guardrails.** Make behavior inspectable (traces, structured logs) and safe (grounded in reported facts and guidance, refusal under uncertainty, no fabricated facts, no diagnosis).

Data & safety ground rules

NON-NEGOTIABLE

Use only **synthetic or public data** — never real PHI. The copilot must not provide a diagnosis or specific medication/dosing advice, must include appropriate disclaimers, and must bias toward caution. Do not copy any proprietary triage protocol verbatim; author your own synthetic guidance modeled on the general structure.

Helpful public / synthetic sources you may draw on:

Source	Useful for
MedlinePlus / WHO fact sheets	Plain-language symptom and condition background to author realistic synthetic guidance
CDC public materials	General symptom / when-to-seek-care guidance to model your protocols on
Synthea	Synthetic patient profiles and histories (no PHI)
Your own synthetic cases	Hand-authored patient messages and labels for your eval set

Briefly note in your README which sources you used and that all patient data is synthetic.

Using AI coding tools — encouraged, and we want to hear about it

You may (and probably should) use AI coding assistants and agents. We build with these tools and we hire people who are excellent with them. **This is a plus, not a minus.** In your writeup, briefly tell us:

- Which tools you used.
- What you delegated to them versus drove yourself.
- Anything you built to make the AI more effective — custom prompts or instructions, tooling, an MCP server, eval loops, scaffolding, etc.
- A moment where the AI led you astray and how you caught it.

Stretch goals (optional — pick what showcases your strengths)

Only if you have time. We would rather see one of these done well than all of them attempted.

- A thin **chat UI** for the patient, plus a structured **clinician view** of the intake summary.
- A **hybrid safety net**: a deterministic red-flag detector working alongside the LLM, with measured trade-offs.
- A **simulated-patient test harness** (an LLM playing patients) to evaluate full conversations at scale.
- A **local / on-prem small-model path** for privacy-sensitive steps, benchmarked against a hosted model.

- A **cost / latency benchmark** across model choices for a chat workload.
- **Multilingual intake**, or a configurable care-level taxonomy for different health systems.

Deliverables

1. **Code** — a public GitHub repository.
2. **README** — setup and run in one or two commands, plus how to exercise the system.
3. **Architecture writeup** (≤2 pages) — your design, your orchestration approach, your safety design, and the key trade-offs you made and why. Include a simple diagram.
4. **Decision log** — the product and technical decisions you made where the brief was ambiguous, and what you deliberately left out.
5. **Evaluation** — your eval set, what you measured (call out under-triage / missed emergencies), the results, and how you would extend it.
6. **"If this were going to production"** (≤1 page) — scale, cost, safety, monitoring, compliance, and what you would do next.
7. **AI-tooling note** — as described above.
8. **Optional:** a 3–5 minute demo video.

What the technical discussion will cover

So you can prepare — the deep-dive will focus on your thinking, not trivia:

- A walk-through of your architecture and why you chose it.
- Safety and failure modes: what happens when it misses a red flag, how you prevent it, and how you would know.
- Trade-offs: model selection, agent design, build-vs-buy, where you would cut scope under a deadline.
- Scaling and cost: how the system behaves and what it costs at volume.
- A small live extension or change to your code.

Ground rules & logistics

- **Time-box:** ~8–12 hours. Judgment and quality over completeness.
- **Ask questions.** A senior engineer surfaces ambiguity early — ask us questions at any point; reasonable assumptions documented in your decision log are equally welcome.
- **Deadline:** please submit within **4–5 days** of receiving this brief.
- **Submission:** share your public repo link with us.

FINALLY

We are genuinely excited to see how you think. Build the slice you would be proud to demo, and tell us the story of the decisions behind it — especially the ones that keep patients safe.